

APEXFORGE

Trading System Pre-Live Validation Playbook

38-stage research, robustness, risk and deployment framework

Designed for MT5 backtests, ORB systems and systematic strategy research

Core principle

A profitable backtest is only evidence of historical profitability. Before risking real capital, separately verify data integrity, statistical edge, drawdown survivability, parameter stability, out-of-sample behavior, execution robustness and live implementation.

Version 1.0 | September 2026

How to use this playbook

- Run the stages in order. Do not skip directly to Sharpe, CAGR or a beautiful equity curve before proving the backtest itself is trustworthy.
- Treat the thresholds as practical research heuristics rather than universal laws. Strategy frequency, asset, holding period, costs and account constraints change what is acceptable.
- Record every result in the same dashboard so future strategies can be compared on identical definitions.
- Prefer stable parameter regions and repeatable behavior over the single best historical parameter combination.
- Do not “repair” every losing year or bad month. A real edge can have drawdowns. The objective is to prove that losses are survivable and statistically consistent with the strategy.

Suggested decision rule

Use hard gates for integrity, out-of-sample survival, execution feasibility and risk-of-ruin. Use softer scoring for ratios such as Sharpe, Sortino and percentage of positive months. A strategy should not pass merely because its average score is high if a critical gate fails.

Recommended evidence hierarchy

Evidence level	What it proves	Strength
In-sample backtest	The idea can fit historical data	Low by itself
Parameter plateau	The result is not dependent on one exact input	Moderate
Out-of-sample	The rules survived unseen history	High
Walk-forward	The process survived multiple unseen windows	High
Monte Carlo / stress tests	The system can survive worse sequencing and execution	High
Demo / paper forward	The implementation behaves correctly in real time	Very high
Small live deployment	Real fills, slippage and operational behavior are acceptable	Highest practical pre-scale evidence

The five questions every system must answer

1. Is the backtest mechanically correct and realistic?
2. Is there a statistically meaningful edge after costs?
3. Can the account survive the worst plausible drawdown and losing sequence?
4. Does the edge remain when parameters, periods and execution assumptions change?
5. Does forward/live behavior remain compatible with the historical model?

38-stage validation map

Block	Stages	Focus
Foundation	1-6	Backtest integrity, sample size, profitability, expectancy, costs, win rate/payoff
Risk & return quality	7-13	Drawdown, recovery, Sharpe, Sortino, Calmar, CAGR
Equity & consistency	14-22	Curve shape, trade dependency, tails, losing streaks, month/year/day/time/side analysis
Robustness	23-32	Regimes, parameters, out-of-sample, walk-forward, Monte Carlo, ruin, slippage, spread, delay, missed trades
Generalization & deployment	33-38	Other instruments, recency, demo, tiny live, sizing, portfolio correlation

For your Gold ORB research

Finish the parameter experiment first, then freeze the candidate rules. From Stage 25 onward, do not keep changing parameters in response to validation results; otherwise the “validation” data becomes new training data.

STAGE 01 | PRE-LIVE VALIDATION

Verify the backtest itself

Prove that the historical result is mechanically valid before trusting any performance metric.

Why it matters

Every later ratio assumes the trades are correct. A strategy with PF 1.8 is meaningless if the EA uses future data, the session clock is wrong, commissions are omitted, or fills occur at prices that would not have been available.

How to check it

- Confirm the exact data source, symbol specification, tick model, spread model, commission and swap settings.
- Verify broker/server timezone and daylight-saving-time handling. This is critical for NY-session ORB because the same server time can correspond to a different New York time after DST changes.
- Inspect a random sample of at least 20-30 trades manually against the chart: opening-range construction, signal candle, entry, stop, target, cancellation rules and exit.
- Confirm no look-ahead: the EA cannot use a candle high/low/close before that candle is completed unless the logic explicitly works intrabar.
- Check for missing bars, abnormal gaps, duplicate orders, partial-fill handling and symbol contract-size changes.

Calculation / definition

No single formula. This is an audit gate.

Worked example

Example

If your ORB is defined from 09:30-10:00 New York time, pick dates before and after the US DST switch. Confirm the range always uses the intended local market window. Then verify MT5 commission, spread and stop execution are included.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Manual trade audit matches rules; costs and timestamps are realistic; no look-ahead or data defects.	Small unexplained discrepancies or uncertain cost model.	Future information, wrong session, impossible fills, or major missing costs.

What ApexForge should store

Store data source, broker timezone, modeling mode, spread/commission assumptions, symbol properties, test dates and an Integrity = PASS/WATCH/FAIL field.

ORB-specific note

For ORB, session-time correctness is a hard gate. Validate New York time and DST before trusting any result.

STAGE 02 | PRE-LIVE VALIDATION

Minimum sample size

Ensure the edge is supported by enough independent trade observations to be worth evaluating.

Why it matters

A 70% win rate over 30 trades can easily be luck. More trades reduce uncertainty, but trade count must also be distributed across years and regimes rather than concentrated in one short period.

How to check it

- Count closed trades after excluding technical duplicates and non-strategy transactions.
- Show trades per year and per quarter.
- Check whether a few high-frequency months dominate the sample.
- For systems that trade rarely, compensate with a longer historical period and stronger out-of-sample evidence.

Calculation / definition

Sample size = number of completed, independent strategy trades.

Worked example

Example

A Gold ORB backtest with 520 trades from 2019-2026 is much more informative than 85 trades from 2024-2026. But 520 trades are still weaker if 300 occurred during one unusual volatility year.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Prefer 300-500+ trades for an intraday ORB; 500+ is strong.	100-300 trades: usable but uncertainty remains high.	<100 trades unless the strategy is inherently low-frequency and spans many years.

What ApexForge should store

Trade count, trades/year, trades/month, median trades/month, percentage of trades in busiest year.

STAGE 03 | PRE-LIVE VALIDATION

Basic profitability: Profit Factor

Measure whether gross winning dollars meaningfully exceed gross losing dollars.

Why it matters

Profit Factor (PF) is intuitive and robust for first-pass evaluation, but it says nothing about sequencing, drawdown duration or overfitting. A PF of 1.5 across 600 trades is generally more convincing than PF 2.3 across 45 trades.

How to check it

- Calculate PF for the full sample.
- Recalculate by year, long/short side and out-of-sample period.
- Confirm PF remains above 1 after realistic costs and stress assumptions.

Calculation / definition

$$\text{Profit Factor} = \text{Gross Profit} / |\text{Gross Loss}|$$

Worked example

Example

Gross winning P&L = \$78,000 and gross losing P&L = -\$52,000. $PF = 78,000 / 52,000 = 1.50$.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Practical target: $PF \geq 1.30$ after costs; 1.50+ is strong.	PF 1.10-1.30: edge may be too thin or require exceptional execution.	$PF \leq 1.0$, or PF collapses below ~1.1 with modest cost stress.

What ApexForge should store

PF full sample, PF by year, PF OOS, PF long/short, stressed PF.

STAGE 04 | PRE-LIVE VALIDATION

Expectancy per trade

Quantify the average edge produced by one trade and express it in risk units when possible.

Why it matters

Expectancy combines win rate and payoff into one number. Expressing it in R makes strategies with different lot sizes and account sizes easier to compare.

How to check it

- Calculate money expectancy and R expectancy.
- Use median R as a secondary check because a few outliers can lift the mean.
- Compare expectancy in-sample, OOS and recent windows.

Calculation / definition

Expectancy = (Win% x Avg Win) - (Loss% x Avg Loss). In R: use wins/losses measured in initial-risk units.

Worked example

Example

Win rate 43%, average win 1.8R, loss rate 57%, average loss 1R. Expectancy = $0.43 \times 1.8 - 0.57 \times 1.0 = +0.204\text{R/trade}$.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Prefer $> +0.10\text{R/trade}$ after costs; $+0.20\text{R}$ or more is strong.	$+0.03\text{R}$ to $+0.10\text{R}$: thin edge, cost-sensitive.	≤ 0 after realistic costs or negative OOS expectancy.

What ApexForge should store

Mean expectancy (\$), mean expectancy (R), median R, rolling 50-trade expectancy.

STAGE 05 | PRE-LIVE VALIDATION

Average trade vs transaction costs

Check that the statistical edge is large enough to survive spread, commission and slippage.

Why it matters

Intraday breakout systems can look profitable when backtest friction is understated. If average gross profit per trade is only slightly larger than trading costs, live slippage can erase the edge.

How to check it

- Estimate average all-in cost per round trip: commission + spread impact + typical slippage + financing if relevant.
- Compare cost with average gross trade edge.
- Stress costs by +25%, +50% and +100%.

Calculation / definition

Cost cushion = Average gross edge per trade / Average all-in transaction cost.

Worked example

Example

Average gross edge = \$24/trade. Normal all-in cost = \$8. Cost cushion = 3.0x. At +50% costs (\$12), the system still has a meaningful net edge.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Prefer edge at least ~2-3x normal costs and profitable at +50% cost stress.	1.5-2x cost cushion.	Edge is close to costs or a small spread/slippage increase removes profitability.

What ApexForge should store

Average gross trade, average net trade, estimated cost/trade, cost cushion, PF at 1.25x/1.5x/2x costs.

ORB-specific note

ORB enters during active periods; assume transaction costs can be worse exactly when the signal occurs.

STAGE 06 | PRE-LIVE VALIDATION

Win rate and payoff ratio

Understand how the system earns money instead of treating win rate as a quality score.

Why it matters

A breakout system may win only 40-45% of the time but make much more on winners than it loses on losers. Conversely, a 75% win-rate strategy can be fragile if losers are several times larger than winners.

How to check it

- Calculate win rate, average win, average loss and payoff ratio.
- Plot distribution of R outcomes.
- Compare long vs short and yearly payoff characteristics.

Calculation / definition

$$\text{Payoff Ratio} = \text{Average Winning Trade} / |\text{Average Losing Trade}|$$

Worked example

Example

Win rate = 41%; Avg win = 2.0R; Avg loss = 0.95R. Payoff = 2.11. Expectancy remains positive despite more losing trades than winners.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
No universal win-rate threshold. Require a combination that produces healthy positive expectancy.	High win rate with payoff <0.5, or low win rate with payoff too small to compensate.	Win rate/payoff combination yields zero or negative expectancy after costs.

What ApexForge should store

Win%, loss%, avg win R, avg loss R, payoff, breakeven win rate, actual minus breakeven win-rate margin.

STAGE 07 | PRE-LIVE VALIDATION

Maximum drawdown depth

Measure the largest historical peak-to-trough capital decline and judge whether it is survivable.

Why it matters

Return without drawdown context is misleading. A 35% CAGR with 40% max DD may be much less attractive than 22% CAGR with 10% DD, especially under prop-firm or psychological constraints.

How to check it

- Calculate both money and percentage max DD from the equity curve, not only balance if open P&L matters.
- Measure DD at the intended live risk level.
- Compare historical DD with Monte Carlo DD later rather than sizing to history alone.

Calculation / definition

$$\text{Drawdown \%} = (\text{Equity Peak} - \text{Subsequent Equity Trough}) / \text{Equity Peak} \times 100$$

Worked example

Example

Equity peaks at \$120,000 and later falls to \$105,600. DD = 14,400 / 120,000 = 12%.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
As a research heuristic: <15% is strong; 15-20% can be acceptable depending on objectives.	20-25% or near account/prop limits.	>25-30% for a system intended to be robust, or any DD that violates the deployment account rules.

What ApexForge should store

Max balance DD, max equity DD, DD date range, DD in R, intended risk-adjusted DD.

ORB-specific note

If targeting prop accounts, size for the firm's daily and total drawdown limits, not merely a personal 20% tolerance.

STAGE 08 | PRE-LIVE VALIDATION

Drawdown duration and recovery time

Measure how long the strategy can remain underwater, not merely how deep the drawdown becomes.

Why it matters

A strategy with only 9% max DD can still be extremely difficult to hold if it spends 18 months below its prior high. Duration also reveals regime dependence and capital inefficiency.

How to check it

- Measure time from equity peak until a new equity high.
- Calculate maximum and median drawdown duration.
- Count how often drawdowns exceed 1, 3, 6 and 12 months.

Calculation / definition

Drawdown duration = date of recovery to prior peak - date the prior peak was established.

Worked example

Example

The system reaches a high on 12 Feb, bottoms at -8.5% in May, and exceeds the old high on 20 September. Depth = 8.5%, duration ~7 months.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Duration is compatible with strategy frequency and trader/account tolerance.	Long stagnation relative to expected trade frequency; repeated 6-12 month underwater periods.	Multi-year stagnation with weak recent evidence or recovery dependent on one exceptional trade.

What ApexForge should store

Max DD duration, median DD duration, current underwater duration, time-to-recovery histogram.

STAGE 09 | PRE-LIVE VALIDATION

Recovery Factor

Compare total net profit with the worst historical drawdown required to achieve it.

Why it matters

Recovery Factor helps distinguish a smooth wealth generator from a system that earns modest profit by accepting very large drawdowns.

How to check it

- Use a consistent definition of net profit and max drawdown.
- Compare full sample, OOS and recent periods.
- Do not rely on it alone if the test duration differs materially between strategies.

Calculation / definition

$$\text{Recovery Factor} = \text{Net Profit} / \text{Maximum Drawdown (same currency units)}$$

Worked example

Example

Net profit = \$60,000; maximum drawdown = \$24,000. Recovery Factor = 2.5.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
>1.5 is a useful target; >2 is strong.	1.0-1.5.	<1, meaning the worst drawdown is as large as or larger than total net profit.

What ApexForge should store

Recovery Factor full/OOS, net profit, max DD currency.

STAGE 10 | PRE-LIVE VALIDATION

Sharpe Ratio

Measure average excess return relative to total return volatility.

Why it matters

Sharpe rewards smoother return streams but can understate the attractiveness of positively skewed breakout systems because upside volatility is penalized too. Use it as one lens, not the final verdict.

How to check it

- Calculate from periodic strategy returns (commonly daily or monthly), not raw trade P&L without normalization.
- Annualize consistently.
- Use the same risk-free-rate convention across systems.

Calculation / definition

Sharpe = (Mean periodic return - risk-free rate) / Standard deviation of periodic returns, annualized.

Worked example

Example

If daily mean excess return and volatility annualize to 18% and 14%, Sharpe is about 1.29.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Sharpe >1 is a practical positive target; >1.5 is very good.	0.5-1.0 can still be viable for certain asymmetric systems.	Persistently <0.5 with poor drawdown- adjusted metrics.

What ApexForge should store

Sharpe full sample, OOS, rolling 6/12-month Sharpe and definition/frequency used.

STAGE 11 | PRE-LIVE VALIDATION

Sortino Ratio

Measure return relative to harmful downside volatility rather than penalizing upside volatility.

Why it matters

Sortino is often more intuitive for trading strategies because large positive returns do not increase the penalty. It is especially useful alongside Sharpe for breakout/trend systems.

How to check it

- Define target/minimum acceptable return consistently (often 0).
- Calculate downside deviation from periodic returns below the target.
- Compare with Sharpe to understand skew.

Calculation / definition

Sortino = (Mean return - target return) / Downside deviation, annualized.

Worked example

Example

Annualized excess return 18%; annualized downside deviation 9%. Sortino = 2.0.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
>1.5 is a useful target; >2 is strong.	1.0-1.5.	<1 combined with weak drawdown metrics.

What ApexForge should store

Sortino full/OOS, downside deviation, rolling Sortino.

STAGE 12 | PRE-LIVE VALIDATION

Calmar Ratio

Compare annualized growth with maximum drawdown.

Why it matters

Calmar directly answers a practical question: how much annualized return did the strategy deliver per unit of worst drawdown? It is very useful when comparing systems intended for the same capital pool.

How to check it

- Use CAGR over a sufficiently long test and percentage max DD.
- Be cautious with short tests where CAGR is unstable.

Calculation / definition

$$\text{Calmar Ratio} = \text{CAGR} / \text{Maximum Drawdown \%}$$

Worked example

Example

CAGR = 24%, Max DD = 12%. Calmar = 2.0.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
>1 good; >2 excellent.	0.5-1.0.	<0.5 unless there is a compelling portfolio diversification benefit.

What ApexForge should store

CAGR, max DD%, Calmar full sample and OOS.

STAGE 13 | PRE-LIVE VALIDATION

CAGR and return quality

Measure annualized compounding while preventing high return from hiding excessive risk.

Why it matters

CAGR is useful for comparing multi-year strategies, but it should always be read beside drawdown, Calmar, leverage and risk-per-trade. High CAGR can often be manufactured simply by increasing position size.

How to check it

- Calculate CAGR from starting and ending equity over exact years.
- Compare CAGR to annualized volatility and max DD.
- Normalize strategy comparisons to similar risk where possible.

Calculation / definition

$$\text{CAGR} = (\text{Ending Equity} / \text{Starting Equity})^{(1 / \text{Years})} - 1$$

Worked example

Example

\$100,000 grows to \$220,000 over 5 years. $\text{CAGR} = (2.2)^{(1/5)} - 1 \approx 17.1\%$.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
No universal minimum. Favor attractive CAGR with Calmar >1 and tolerable DD.	High CAGR produced only through aggressive leverage or one exceptional year.	CAGR is low relative to drawdown/risk and offers little portfolio benefit.

What ApexForge should store

CAGR, annualized return, annualized volatility, return/DD ratio, risk-normalized comparison.

STAGE 14 | PRE-LIVE VALIDATION

Equity curve shape and path quality

Visually and statistically inspect how the profits were earned through time.

Why it matters

Two systems can have identical net profit and PF but radically different paths. Smoothness is not everything, yet long flat periods, sudden one-off jumps or late-stage degradation often reveal dependence on a small set of conditions.

How to check it

- Plot balance and mark-to-market equity.
- Overlay drawdown under the equity curve.
- Inspect linear and log scales.
- Annotate major profit jumps and drawdowns.
- Calculate rolling 50-trade or 3/6-month expectancy.

Calculation / definition

No single formula. Combine visual path inspection with rolling metrics and concentration measures.

Worked example

Example

If 70% of total profit appears in two vertical jumps during 2020 and the rest of the curve is flat, the headline CAGR is not representative of a stable edge.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Profit accumulates across multiple periods with understandable drawdowns.	Long stagnation, clustering or recent flattening.	Most profit comes from isolated jumps or a small historical window.

What ApexForge should store

Equity/balance chart, underwater chart, rolling PF, rolling expectancy, rolling Sharpe.

ORB-specific note

Mark the largest breakout days on the curve. You want to know whether the strategy requires rare trend explosions to make the entire backtest.

STAGE 15 | PRE-LIVE VALIDATION

Best-trade dependency

Test whether a handful of exceptional winners are responsible for most of the strategy's edge.

Why it matters

Positive-skew breakout systems naturally have important winners, so removing top trades will hurt them. The question is whether the entire system collapses when only a tiny fraction of trades are removed.

How to check it

- Recalculate metrics after removing best 1, 5, 10 trades.
- Also remove the top 1%, 2% and 5% of trades by P&L/R.
- Measure the percentage of total profit contributed by the top 5% of winners.

Calculation / definition

Top-trade concentration = Profit from top X% trades / Total net profit.

Worked example

Example

Original net profit = \$80k. Removing top 10 trades leaves \$61k and PF 1.28: acceptable dependence. If profit falls to \$7k and PF 1.03, the edge is extremely concentrated.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
System remains meaningfully profitable after reasonable top-trade removal.	Large degradation but still positive; may be normal for trend systems and needs context.	A few trades explain nearly all net profit.

What ApexForge should store

Net/PF after remove top 1/5/10 and 1%/5%; top-5%-profit contribution.

STAGE 16 | PRE-LIVE VALIDATION

Worst-trade and tail-risk analysis

Understand losses that are much worse than the “normal” stop and identify gap/slippage tail risk.

Why it matters

Backtests can hide operational tail events. A strategy designed for -1R stops may occasionally realize -3R during gaps or fast markets. Those events determine capital and prop-rule survivability.

How to check it

- List worst 10 trades in R and currency.
- Compare realized loss to intended stop loss.
- Review worst day/week/month.
- Inspect whether losses cluster around news, market open or illiquidity.

Calculation / definition

Tail loss multiple = Worst realized loss / Intended initial risk.

Worked example

Example

Normal risk is \$500 per trade. Worst historical realized loss is \$1,650 = 3.3R because of a gap. Risk sizing must account for that possibility.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Tail events are understood and still survivable.	Occasional 2-3R losses requiring lower size or event controls.	Unbounded losses, unexplained gaps, or one event can violate account limits.

What ApexForge should store

Worst 10 trades, worst day/week/month, tail-loss multiple, 95th/99th percentile loss.

STAGE 17 | PRE-LIVE VALIDATION

Consecutive losses and losing streaks

Estimate the operational and psychological stress the strategy can impose before recovering.

Why it matters

A strategy with positive expectancy can still produce long losing sequences. Live streaks can exceed the historical maximum, so position sizing based on exactly the historical worst streak is unsafe.

How to check it

- Calculate max consecutive losing trades and max consecutive losing days.
- Measure distribution of streak lengths.
- Use Monte Carlo later to estimate the 95th/99th percentile streak.

Calculation / definition

Losing streak = maximum number of consecutive closed trades with negative net P&L.

Worked example

Example

Historical worst streak = 8 losses. At 0.5% risk/trade, eight full-R losses imply roughly 4% loss before compounding. Plan for a plausible 12-14 loss run, not exactly eight.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Expected extended streak is comfortably survivable at planned risk.	Historical streak already stresses account/prop rules.	A plausible streak could trigger ruin or forced strategy abandonment.

What ApexForge should store

Max streak, 95th percentile streak (Monte Carlo), loss-days streak, drawdown caused by streak.

STAGE 18 | PRE-LIVE VALIDATION

Monthly return consistency

See whether profitability is broadly distributed through time and quantify bad-month behavior.

Why it matters

Monthly analysis reveals seasonality, regime dependence and whether one or two months drive annual results. The percentage of winning months is useful, but not a universal target for lower-frequency strategies.

How to check it

- Create monthly return heatmap.
- Calculate positive-month percentage, median month, best/worst month and monthly standard deviation.
- Review clusters of consecutive losing months.

Calculation / definition

Monthly return = (Month-end equity / Prior month-end equity) - 1, using a consistent mark-to-market convention.

Worked example

Example

Over 84 months, 53 are positive = 63.1%. Worst month = -4.8%, best = +7.2%, median = +0.9%.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
For a frequent intraday system, ~60%+ positive months is encouraging, with manageable worst months.	50-60% can still work if payoff is strong.	Profit depends on a small number of extraordinary months and most months are flat/negative.

What ApexForge should store

Monthly heatmap, % positive, median/mean, best/worst, consecutive negative months.

STAGE 19 | PRE-LIVE VALIDATION

Year-by-year performance

Confirm that the system is not simply a one-regime or one-year phenomenon.

Why it matters

A backtest can have attractive aggregate metrics while one year creates nearly all gains. Year-level analysis is one of the fastest ways to expose regime dependence and deterioration.

How to check it

- Show net return, PF, expectancy, DD and trade count for every year.
- Calculate the contribution of the best year to total profit.
- Inspect recent years separately.

Calculation / definition

Best-year contribution = Profit in best year / Total net profit.

Worked example

Example

Returns of +14%, +18%, +9%, +16%, +11% are more convincing than -4%, +1%, +68%, -7%, +2% even if the final CAGR is similar.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Multiple profitable years; losing years are limited and explainable; no single year dominates.	One unusually strong year contributes >40-50% of total profit.	Most years lose and one regime creates the backtest.

What ApexForge should store

Year table: return, net P&L, PF, expectancy, max DD, trades, profit contribution.

STAGE 20 | PRE-LIVE VALIDATION

Day-of-week analysis

Identify whether edge or losses systematically cluster on particular weekdays.

Why it matters

ORB behavior can differ around Monday re-pricing, Friday positioning or scheduled macro releases. However, removing a “bad day” after observing it can easily become data mining.

How to check it

- Calculate trades, PF, expectancy and win rate by weekday.
- Use confidence intervals or at least minimum trade-count rules before acting on differences.
- Validate any weekday filter out-of-sample.

Calculation / definition

Same core metrics, grouped by entry weekday.

Worked example

Example

Friday PF = 0.92 over 31 trades is weak evidence. Friday PF = 0.91 over 220 trades across 7 years deserves investigation.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
No catastrophic weekday dependence, or any filter has strong rationale and OOS validation.	One day is weak but sample is small.	System profitability depends on a single weekday or proposed filter only works in-sample.

What ApexForge should store

Weekday table and chart with trade count, PF, expectancy R, return contribution and OOS comparison.

ORB-specific note

Do not remove a weekday just because its in-sample PF is low. Require adequate trades and OOS confirmation.

STAGE 21 | PRE-LIVE VALIDATION

Time-of-day and entry-timing analysis

Locate when the edge exists and verify the strategy is not accidentally exploiting a timing artifact.

Why it matters

ORB is explicitly time-dependent. Small clock errors, DST mistakes or delayed entries can materially change results. Time-bucket analysis also helps explain whether the edge is concentrated immediately after the open or later.

How to check it

- Group by entry hour/minute and minutes after opening-range completion.
- Compare expected signal time with actual fill time.
- Test modest delays separately in Stage 31.

Calculation / definition

Group trade metrics by local market time or elapsed minutes from session event.

Worked example

Example

Entries 0-10 minutes after range completion show PF 1.48; 10-30 minutes show PF 1.21; >30 minutes PF 0.96. This suggests the breakout edge decays with time, but the pattern must be OOS-validated.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Timing behavior is logical, stable and correctly aligned to market time.	Sharp timing cliff with small sample.	Performance disappears when timestamps are corrected or shifted slightly.

What ApexForge should store

Entry-time heatmap, elapsed-time buckets, PF/expectancy by bucket, DST audit flag.

ORB-specific note

This stage is especially important for ORB because timing is part of the strategy definition, not a cosmetic filter.

STAGE 22 | PRE-LIVE VALIDATION

Long vs short decomposition

Determine whether both sides contribute or the system is merely expressing a directional market bias.

Why it matters

Gold, indices and FX can have persistent long/short asymmetries. That is not automatically bad, but you need to know whether “ORB edge” is actually just a long-only bull-market effect.

How to check it

- Calculate trade count, PF, expectancy, DD and profit contribution separately for longs and shorts.
- Compare performance across bull/bear periods.
- Re-test OOS by side.

Calculation / definition

Same strategy metrics grouped by direction.

Worked example

Example

Long PF 1.56 over 310 trades; short PF 1.27 over 280 trades: both contribute. If short PF is 0.94, the strategy may need a structural explanation rather than silently relying on longs.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Both sides are profitable, or one-side design is intentional and robust.	One side weak with limited sample.	Aggregate edge disappears once long-term market drift is removed.

What ApexForge should store

Long/short PF, expectancy, DD, trades and contribution; regime-conditioned side analysis.

STAGE 23 | PRE-LIVE VALIDATION

Market regime analysis

Identify the environments in which the strategy makes and loses money.

Why it matters

A robust system does not need to win in every regime, but you should understand its exposures. ORB strategies often respond strongly to volatility, range compression/expansion and trend conditions.

How to check it

- Segment trades by ATR/volatility percentile.
- Classify trend vs range using a rule defined before inspecting results.
- Review bull/bear, high/low volatility and major macro-event periods.
- Avoid inventing dozens of regime filters until only profitable cells remain.

Calculation / definition

Metrics are grouped by pre-defined regime labels; compare expectancy and PF across groups.

Worked example

Example

Low-volatility quartile PF 1.05, middle 50% PF 1.35, high-volatility quartile PF 1.62. This supports a volatility-related edge and informs later stress expectations.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Edge has understandable regime behavior and remains viable across more than one narrow state.	One regime dominates, but exposure is intentional and diversifiable.	System only works in a tiny historically rare regime selected after the fact.

What ApexForge should store

Regime matrix: trades, PF, expectancy, return, DD and share of total P&L by regime.

ORB-specific note

Your ATR/range filter is effectively a regime hypothesis. Validate that the mechanism behaves consistently rather than optimizing only the best volatility bucket.

STAGE 24 | PRE-LIVE VALIDATION

Parameter stability and plateaus

Prove that performance does not depend on one precise “magic” parameter.

Why it matters

A true market effect should usually tolerate nearby parameter values. A sharp isolated peak often indicates fitting to historical noise. Choose robust neighborhoods, not the maximum backtest result.

How to check it

- Sweep one parameter across a sensible range while keeping logic unchanged.
- Visualize PF, expectancy, DD and net return versus parameter value.
- For two important parameters, create a heatmap/surface and look for broad good regions.
- Prefer a central value inside the plateau.

Calculation / definition

No single ratio. Stability = similar acceptable results across neighboring parameter values.

Worked example

Example

ATR filter: 0.40 PF 1.38, 0.45 PF 1.44, 0.50 PF 1.49, 0.55 PF 1.43, 0.60 PF 1.36 = healthy plateau. If only 0.50 gives PF 1.8 while neighbors are ~1.0, treat it as overfit.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Broad plateau; candidate value chosen away from fragile boundary.	Moderate sensitivity requiring more OOS evidence.	Needle peak / cliff-like sensitivity.

What ApexForge should store

Parameter-vs-metric charts, 2D heatmaps, plateau width, rank stability across years.

ORB-specific note

For your ATR range-filter variants, prefer the middle of a broad stable band even if another setting has slightly higher net profit.

STAGE 25 | PRE-LIVE VALIDATION

Out-of-sample (OOS) test

Test frozen rules on historical data that was not used to choose the parameters.

Why it matters

This is one of the strongest defenses against overfitting. The important rule is procedural: once you inspect OOS, do not repeatedly tune against it or it stops being out-of-sample.

How to check it

- Split history chronologically (for example 70% development / 30% OOS) or reserve the most recent years.
- Finalize rules and parameters using development data only.
- Run once on OOS and compare PF, expectancy, DD and trade frequency.

Calculation / definition

OOS retention = OOS metric / In-sample metric (use carefully; some metrics are nonlinear).

Worked example

Example

IS PF 1.55, OOS PF 1.36, both positive expectancy: credible. IS PF 1.80, OOS PF 1.02: likely overfit or regime-dependent.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
OOS remains meaningfully profitable; quality may be lower but consistent with the same mechanism.	OOS marginal but positive, requiring more data/forward evidence.	OOS negative or strategy behavior materially changes.

What ApexForge should store

IS vs OOS table; OOS flag; parameter lock date; OOS equity curve.

ORB-specific note

Freeze the chosen ORB parameter set before viewing the reserved OOS segment.

STAGE 26 | PRE-LIVE VALIDATION

Walk-forward testing

Repeat the unseen-data test across multiple sequential windows instead of relying on one split.

Why it matters

One lucky OOS period can still mislead. Walk-forward analysis asks whether the research process and parameter choice repeatedly survive future windows.

How to check it

- Choose a fixed development window and subsequent test window.
- Repeat through history without leaking future data.
- Track how many OOS windows are profitable and whether parameters jump wildly.

Calculation / definition

Walk-forward efficiency and consistency can be summarized, but the core evidence is the series of independent OOS windows.

Worked example

Example

Train 2018-2020 -> test 2021; train 2019-2021 -> test 2022; continue. If 5 of 6 test windows are positive with similar behavior, confidence rises substantially.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Majority of OOS windows profitable with manageable DD and no unstable parameter chasing.	Mixed windows but aggregate OOS positive.	Most unseen windows fail or parameters change wildly every cycle.

What ApexForge should store

Walk-forward window table, stitched OOS equity curve, % profitable windows, parameter path.

STAGE 27 | PRE-LIVE VALIDATION

Monte Carlo simulation

Estimate what could happen when the same underlying edge experiences a worse trade sequence or execution noise.

Why it matters

Historical order is only one possible sequence. Monte Carlo helps quantify probable drawdown, streak and terminal-equity ranges rather than pretending historical max DD is the worst possible future outcome.

How to check it

- Resample or reshuffle trades in a way consistent with your assumptions.
- Run 1,000-10,000 simulations.
- Measure median, 90th, 95th and 99th percentile max DD, losing streak and ending equity.
- Optionally perturb slippage or trade returns.

Calculation / definition

Simulation output distributions rather than one formula. Key statistic: percentile(max drawdown).

Worked example

Example

Historical max DD = 11%. Across 10,000 reshuffles, median max DD = 13%, 95th = 19%, 99th = 23%. Risk should be sized with the distribution, not 11% alone.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
95th-percentile DD and streak are comfortably survivable; terminal loss probability is low.	Stress percentiles approach account limits.	Meaningful probability of violating loss limits or ending deeply negative.

What ApexForge should store

Simulation count, DD percentiles, streak percentiles, terminal equity fan chart, probability of breach.

ORB-specific note

Use Monte Carlo to convert a historical DD number into a distribution of plausible future DDs.

STAGE 28 | PRE-LIVE VALIDATION

Risk of ruin

Quantify the probability that the chosen position sizing causes an unacceptable capital loss or account breach.

Why it matters

A positive edge does not guarantee survival if leverage is too high. Ruin should be defined operationally: account loss threshold, prop-firm max loss, or a personal stop level.

How to check it

- Define “ruin” explicitly (for example 30% account loss, or 10% prop max drawdown).
- Estimate via Monte Carlo at different risk-per-trade levels.
- Include tail-loss assumptions and correlated same-day trades if relevant.

Calculation / definition

Prefer simulation-based probability of breaching the defined ruin threshold rather than relying on simplified closed-form formulas.

Worked example

Example

At 1% risk/trade, 4.2% of simulations breach a 20% loss threshold. At 0.4% risk, only 0.1% breach. The lower risk is much more defensible.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Ruin/breach probability effectively near zero for the intended use.	Small but non-trivial breach probability.	Material probability of account/prop failure under plausible sequences.

What ApexForge should store

Ruin definition, risk/trade, breach probability, risk-vs-breach curve.

STAGE 29 | PRE-LIVE VALIDATION

Slippage stress test

Test whether the strategy remains profitable when fills are worse than the ideal backtest assumption.

Why it matters

Breakout entries often occur during fast movement, exactly when slippage can increase. A system that requires perfect stop fills may not survive live execution.

How to check it

- Apply realistic baseline slippage.
- Repeat at 1.5x, 2x and 3x baseline or add fixed adverse ticks/points.
- Stress entries and exits separately if possible.

Calculation / definition

Stressed net P&L = Original fills adjusted by assumed adverse execution.

Worked example

Example

PF: baseline 1.48, 1.5x slippage 1.39, 2x 1.31, 3x 1.18 = graceful degradation. Baseline 1.48 -> 2x 1.01 is fragile.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Edge degrades gradually and remains viable under plausible bad fills.	Meaningful degradation but still positive.	Small slippage increase destroys the edge.

What ApexForge should store

PF/net/expectancy at baseline, 1.5x, 2x, 3x slippage; break-even slippage.

ORB-specific note

Stop-entry breakout orders can experience adverse slippage during fast expansion candles; model this deliberately.

STAGE 30 | PRE-LIVE VALIDATION

Spread stress test

Measure sensitivity to wider bid-ask spreads, especially around session open and volatile periods.

Why it matters

ORB systems may enter when spreads are temporarily wider. Tester assumptions based on average spreads can materially overstate live results.

How to check it

- Test baseline, +25%, +50% and +100% spread.
- If possible use variable historical spread/tick data.
- Inspect whether losing trades cluster at unusually high spreads.

Calculation / definition

Reprice entries/exits using stressed bid/ask spread assumptions.

Worked example

Example

Base PF 1.51; +50% spread PF 1.35; +100% PF 1.19. Still positive, though lower. If +25% already produces PF 1.04, edge is too thin.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Profitable at +50% spread stress for an intraday strategy is a useful robustness sign.	Only modest cushion.	Normal live spread variability is sufficient to eliminate edge.

What ApexForge should store

PF/net/expectancy by spread multiplier and break-even spread.

ORB-specific note

Session open and news can produce spread spikes. A fixed “normal” spread test is not enough for a breakout system.

STAGE 31 | PRE-LIVE VALIDATION

Entry-delay and price-perturbation test

Check whether the strategy survives small timing and price differences between backtest signal and live execution.

Why it matters

Real systems experience latency, broker routing and tick timing differences. A robust market effect should not require the exact historical tick to remain profitable.

How to check it

- Delay execution by 1, 3, 5 and 10 seconds where data supports it, or by 1+ ticks/bars depending on strategy.
- Apply adverse entry-price perturbation.
- Test signal calculation on adjacent timestamps if appropriate.

Calculation / definition

Compare performance under deterministic entry delays / adverse price offsets.

Worked example

Example

Baseline PF 1.47; +3 sec 1.42; +5 sec 1.35; +10 sec 1.22 = acceptable degradation. If +1 sec changes PF to 1.03, investigate.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Small delay/price noise does not eliminate the edge.	Edge weakens quickly but remains positive.	Requires near-perfect timing/fill to work.

What ApexForge should store

PF/expectancy vs delay, break-even delay, adverse tick sensitivity.

ORB-specific note

A timing-sensitive ORB should degrade gradually, not collapse because the entry happened one or two ticks later.

STAGE 32 | PRE-LIVE VALIDATION

Missed-trade stress test

Verify the system is not dependent on capturing every historical signal.

Why it matters

Live infrastructure can miss trades because of restarts, disconnections, rejected orders or broker issues. Random trade removal also reveals whether a few specific trades are disproportionately important.

How to check it

- Randomly remove 5%, 10% and 20% of trades over many simulation runs.
- Measure distribution of PF, DD and terminal equity after removals.
- Optionally model missed trades concentrated during volatile periods.

Calculation / definition

Monte Carlo subsampling: remove a random fraction of historical trades and recompute metrics.

Worked example

Example

With 10% random trades missed over 5,000 runs, median PF remains 1.39 and 95% of outcomes are profitable. This is much stronger than a strategy that frequently becomes negative.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Most stressed runs remain profitable and within risk limits.	Performance becomes highly variable.	Missing a small fraction of trades commonly turns the strategy negative.

What ApexForge should store

Profitability probability at 5/10/20% missed, PF distribution, DD distribution.

STAGE 33 | PRE-LIVE VALIDATION

Cross-instrument generalization

Test whether the underlying market logic has evidence beyond one exact symbol without over-optimizing each instrument.

Why it matters

A strategy does not need to work everywhere, but similar behavior on related markets can support the idea that you found a market effect rather than noise unique to one backtest.

How to check it

- Apply the same core rules to a small set of suitable instruments.
- Initially avoid custom optimization per symbol.
- Normalize sessions and contract details correctly.
- Document why the mechanism should or should not transfer.

Calculation / definition

Compare risk-normalized PF, expectancy and DD across instruments using unchanged or minimally adapted rules.

Worked example

Example

After Gold ORB is frozen, test NASDAQ, US30 and selected FX pairs with the same conceptual range/filter logic. Even modest positive results can strengthen confidence if the mechanism is coherent.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Reasonable behavior across more than one market, or strong evidence why the edge is asset-specific.	Only original symbol works but robustness evidence is otherwise strong.	Tiny rule changes are required for every symbol and all untouched tests fail.

What ApexForge should store

Instrument comparison table: unchanged-rule result, trade count, PF, expectancy, DD.

STAGE 34 | PRE-LIVE VALIDATION

Recent-performance and edge-decay check

Ensure an excellent long backtest is not hiding a strategy that stopped working recently.

Why it matters

A 10-year result can stay attractive even after two years of deterioration. Recency analysis helps identify structural change, implementation drift or a volatility regime the strategy cannot handle.

How to check it

- Calculate 3, 6, 12 and 24-month rolling metrics.
- Compare recent PF, expectancy, frequency and DD to historical distributions.
- Check whether recent weakness is statistically unusual or still within expected drawdown behavior.

Calculation / definition

Recent z-score / percentile can be used, but simple rolling metrics plus historical percentile bands are often sufficient.

Worked example

Example

Full PF 1.48; last 24 months 1.35; last 12 months 1.28: degradation but still viable. Full PF 1.48; last 24 months 0.92 with normal trade count: major concern.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Recent behavior remains within historical variability and edge stays positive.	Recent deterioration near lower historical percentile.	Sustained recent negative expectancy inconsistent with historical behavior.

What ApexForge should store

Rolling PF, rolling expectancy, rolling DD, current percentile versus history, edge-decay flag.

STAGE 35 | PRE-LIVE VALIDATION

Demo / paper forward test

Validate the real-time EA implementation and execution assumptions without risking meaningful capital.

Why it matters

Backtest robustness does not prove the live code behaves correctly. Forward testing catches timezone, order-state, reconnection, duplicate-order, broker-symbol and real spread/slippage problems.

How to check it

- Freeze code and parameters before starting.
- Run continuously through enough trades/market conditions; for an active strategy aim for roughly 50-100+ trades when practical.
- Log signal time, requested price, actual fill, spread, slippage, stop/target changes and errors.
- Compare forward trade distribution with backtest expectations.

Calculation / definition

Implementation gap = Forward observed metric - Backtest expected metric, interpreted with sampling uncertainty.

Worked example

Example

Backtest predicts average slippage 0.8 point; demo shows 1.1. Trade frequency and signal timestamps match. That is plausible. If demo produces 30% fewer signals, investigate implementation before live.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Signals, trade frequency, costs and state handling are compatible with backtest assumptions.	Small discrepancies explainable by sample size or execution.	Missing/duplicate trades, wrong sessions, or execution materially worse than model.

What ApexForge should store

Forward-vs-backtest trade reconciliation, slippage, spread, missed signals, technical errors.

ORB-specific note

Compare each demo signal with the historical rule logic, not only the final P&L.

STAGE 36 | PRE-LIVE VALIDATION

Tiny live deployment

Observe true broker fills and operational behavior with minimal financial risk before scaling.

Why it matters

Demo environments can differ from live execution. The purpose of tiny live is not to prove profitability again; it is to validate the implementation under real money and real broker conditions.

How to check it

- Use the minimum practical live risk/size.
- Keep the strategy exactly as validated.
- Log fills, rejections, latency, slippage, spread spikes, VPS/network events and manual interventions.
- Define a minimum evidence period before scaling.

Calculation / definition

Live implementation slippage = Actual fill - modeled/expected fill, signed adversely; track distribution.

Worked example

Example

Run at roughly 0.10-0.25% risk/trade (or minimum lot if lower). After sufficient trades, compare median and 95th-percentile slippage with stress assumptions.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Live implementation behaves within stress-tested assumptions.	Costs somewhat higher but still within break-even cushion.	Operational failures, broker behavior or live costs invalidate the model.

What ApexForge should store

Live trade reconciliation, execution stats, error log, planned vs realized risk.

ORB-specific note

Do not increase risk because the first few live trades win. Scale only after the predefined evidence requirement is met.

STAGE 37 | PRE-LIVE VALIDATION

Final risk sizing

Choose position size from survivability evidence rather than from desired return.

Why it matters

Leverage can make almost any profitable backtest look spectacular. Risk sizing should be the output of drawdown distributions, tail losses, losing streaks and account rules.

How to check it

- Start from the maximum acceptable account drawdown or prop loss limit.
- Use Monte Carlo 95th/99th-percentile DD at candidate risk levels.
- Include same-day multiple trades, gap loss and correlated systems.
- Leave a safety buffer instead of sizing to the exact limit.

Calculation / definition

Scale risk iteratively until stress DD remains below the chosen capital-loss limit with sufficient buffer.

Worked example

Example

At 0.5% risk/trade, MC 95% DD = 14%; at 1% risk it is ~27%. If your acceptable DD is 18%, 0.5% is defensible while 1% is not.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
Planned size keeps stress DD and ruin probability comfortably below limits.	Stress DD uses most of the available loss budget.	Sizing chosen mainly to hit a return target or violates stress limits.

What ApexForge should store

Risk/trade scenarios, MC DD percentiles, expected annual return, breach probability, recommended risk band.

STAGE 38 | PRE-LIVE VALIDATION

Portfolio correlation and diversification

Decide whether the validated strategy improves the total portfolio rather than merely adding another version of the same risk.

Why it matters

Your long-term objective is multiple low-correlation systems. Correlation should be measured between strategy return streams, not inferred from whether the traded instruments are different.

How to check it

- Align daily (or suitable periodic) strategy returns.
- Calculate Pearson and, optionally, rank correlation.
- Inspect correlation during drawdowns and stressed periods, not just full-sample average.
- Build combined equity/DD and test portfolio risk allocation.

Calculation / definition

Correlation = covariance(strategy A returns, strategy B returns) / (std A x std B).

Worked example

Example

Two systems trade different instruments but daily-return correlation is 0.72: weak diversification. Another system on the same instrument may have correlation 0.12 because its edge occurs in different regimes.

Decision threshold

PASS	WATCH / INVESTIGATE	FAIL / REDESIGN
<0.30 is excellent diversification; 0.30-0.50 useful; combined portfolio DD improves.	0.50-0.70 moderate overlap.	>0.70 with similar drawdown timing unless the strategy has exceptional standalone value.

What ApexForge should store

Correlation matrix, rolling correlation, drawdown overlap, combined equity curve, portfolio PF/Sharpe/Sortino/Calmar/DD.

ORB-specific note

Once ORB passes, the next system should be evaluated partly on whether it reduces portfolio drawdown during ORB weakness.

Final go / no-go scorecard

Use hard gates first

A high average score must never override a failed integrity, OOS, execution or risk-of-ruin gate. A strategy with beautiful ratios but impossible live fills is a rejection, not a “75/100”.

Gate	Evidence	Type	Required decision
1. Integrity	Backtest mechanics, timing, costs, no look-ahead	HARD	PASS required
2. Evidence	Trade count and multi-year distribution	SOFT/HARD	Enough observations for strategy frequency
3. Edge	PF, expectancy, payoff, cost cushion	HARD	Positive after realistic costs
4. Risk	DD depth/duration, tail, streaks	HARD	Survivable with buffer
5. Return quality	Sharpe, Sortino, Calmar, Recovery, CAGR	SOFT	Prefer balanced risk-adjusted profile
6. Consistency	Monthly/yearly/side/time behavior	SOFT	No single period dependency
7. Robustness	Regimes, parameter plateau, OOS, walk-forward	HARD	OOS positive and parameters stable
8. Stress survival	Monte Carlo, ruin, costs, delay, missed trades	HARD	Plausible stresses remain survivable
9. Implementation	Demo then tiny live	HARD	Forward behavior reconciles
10. Portfolio fit	Correlation and combined DD	SOFT/HARD	Adds diversification or exceptional standalone edge

Suggested classification

Status	Meaning	Action
GREEN - Candidate for live	All hard gates pass; soft metrics are acceptable; tiny live execution validated.	Deploy at conservative risk and monitor predefined live health metrics.
AMBER - More evidence needed	No fatal flaw, but sample/OOS/stress/forward evidence is insufficient or mixed.	Collect more evidence without continuously re-optimizing.
RED - Reject / redesign	One or more hard gates fail, or edge is fragile to modest realistic changes.	Archive the version. Redesign only if there is a new economic/market hypothesis, not just parameter searching.

Compact thresholds dashboard

Metric / Test	Practical target	Caution
Profit Factor	≥ 1.30 after costs	1.10-1.30 thin edge
Expectancy	$> +0.10R/\text{trade}$	$< +0.05R$ fragile

Cost cushion	~2-3x normal costs	<1.5x fragile
Max DD	Prefer <15-20% standalone	Must fit account rules
Recovery Factor	>1.5	<1 weak
Sharpe	>1	0.5-1 context-dependent
Sortino	>1.5	<1 weak
Calmar	>1	<0.5 weak
Positive months	~60%+ encouraging for active system	Not a universal rule
Parameter shape	Broad plateau	Needle peak = overfit risk
OOS	Meaningfully profitable	Negative OOS = hard warning/fail
Monte Carlo	95% DD comfortably survivable	Do not size to historical DD only
Risk of ruin	Near 0% at intended risk	Any meaningful breach probability needs lower risk
Correlation	<0.30 excellent; <0.50 useful	>0.70 often redundant

Recommended order for every new strategy

6. Freeze the exact strategy specification and audit the backtest.
7. Confirm sample size and that trades are distributed across years/regimes.
8. Validate the raw edge: PF, expectancy, payoff and cost cushion.
9. Validate survivability: drawdown depth/duration, tails and losing streaks.
10. Inspect equity curve, month/year consistency, time/side decomposition and regimes.
11. Run parameter stability and choose a plateau value - not the best isolated point.
12. Freeze parameters and run OOS.
13. Run walk-forward or multiple chronological validation windows.
14. Run Monte Carlo, risk-of-ruin and execution stress tests.
15. Check recent edge and cross-instrument generalization where logically appropriate.
16. Run demo/paper forward without changing the system.
17. Run tiny live, reconcile real fills/costs and operational errors.
18. Set final risk from stress drawdown distributions and account constraints.
19. Only then evaluate portfolio correlation and allocate capital across systems.

Anti-overfitting rule

If a validation test fails, do not automatically add a filter that would have fixed that historical failure. First ask whether there is a genuine market mechanism behind the change. Otherwise the validation set becomes part of the optimization loop.

Suggested ApexForge report sections

- Executive verdict and hard-gate status
- Backtest integrity and assumptions
- Core performance ratios
- Drawdown and tail-risk analysis
- Equity curve and rolling metrics
- Monthly / yearly / weekday / time-of-day / long-short breakdowns
- Regime analysis
- Parameter robustness surfaces
- In-sample vs out-of-sample vs walk-forward
- Monte Carlo and risk-of-ruin
- Execution stress tests
- Recent edge-decay monitor
- Forward/live reconciliation
- Portfolio correlation and combined equity
- Final pass/watch/fail decision with reasons

Blank strategy validation record

Field	Result / Notes
Strategy name	
Version	
Instrument / session	
Backtest period	
Development period	
OOS period	
Number of trades	
Profit Factor	
Expectancy (R)	
Cost cushion	
Max equity DD	
Max DD duration	
Recovery Factor	
Sharpe	
Sortino	
Calmar	
CAGR	
Positive months %	
Best-year contribution %	
Top-5%-trade profit contribution	
OOS PF / expectancy	
Walk-forward result	
MC 95% max DD	
MC 99% max DD	
Risk of ruin / breach probability	
Slippage stress result	
Spread stress result	
Entry-delay result	
Missed-trade result	
Recent 12m PF / expectancy	
Demo forward result	
Tiny live execution result	

Final risk per trade	
Correlation to portfolio	
Final verdict	GREEN / AMBER / RED
Reason	

End of playbook

The purpose of validation is not to prove that a system cannot lose. It is to prove that the edge is credible, the losses are survivable, and the implementation remains within the assumptions used to justify risking capital.